

GUIA COMPLETO DO PROJETO

Fabulosa E-commerce - Arquitetura & Decisões Técnicas

Gerado em: 11/08/2026, 10:33:57

GUIA COMPLETO DO PROJETO FABULOSA E-COMMERCE

Arquitetura, Tecnologias e Decisões Técnicas

Este documento explica o projeto Fabulosa E-commerce (Fork: ProjetoFabuloja-fork), detalhando cada tecnologia escolhida, o motivo da escolha e como tudo se conecta.

Data: 11/08/2026, 10:33:57

1. VISÃO GERAL DA ARQUITETURA

O projeto segue uma arquitetura moderna de aplicação full-stack desacoplada:

- Frontend: React 19 + Vite 7 + TypeScript (SPA)
- Backend: Fastify + TypeScript (API REST)
- Database: PostgreSQL + Prisma ORM
- Cache: Redis (sessões, rate limiting futuro)
- Deploy: Docker Compose (dev) / Docker multi-stage (prod)
- CI/CD: GitHub Actions (lint !' typecheck !' build)

Padrão: Frontend e Backend separados, comunicando via API REST + cookies HttpOnly.

2. FRONTEND - TECNOLOGIAS E PORQUÊ

React 19 + TypeScript

- React 19: Concurrent features, Server Components ready, melhor performance
- TypeScript strict mode: Type safety em compile-time, refactoring seguro, autocomplete
- Vite 7: Build ultra-rápido, HMR instantâneo, tree-shaking nativo, ESM moderno

Tailwind CSS 4

- Utility-first: Desenvolvimento rápido, consistência visual, bundle pequeno (purge automático)
- JIT compiler: Build rápido, apenas CSS usado
- Design system: Tokens de cor, spacing, typography centralizados
- Responsive first: Mobile-first nativo, breakpoints intuitivos

React Router DOM 7

- Data loading: loaders/actions para data fetching declarativo
- Type-safe routes: Tipagem automática de params/query
- Lazy loading: Code-splitting automático por rota

Lucide React

- Tree-shakeable: Importa apenas ícones usados
- Consistência: Mesmo estilo visual em todo app
- SVG inline: Sem requests extras, estilável via CSS

Vitest + React Testing Library

- Vitest: Nativo Vite, rápido, ESM nativo, compatível Jest API
- RTL: Testes centrados no usuário (queries por role/label/text)
- happy-dom: DOM leve, rápido, sem JSDOM pesado

3. BACKEND - TECNOLOGIAS E PORQUÊ

Fastify

- Performance: 2x mais rápido que Express, baixo overhead
- Schema validation: Integração nativa com JSON Schema / Zod
- TypeScript first: Tipagem nativa de rotas, hooks, plugins
- Plugin ecosystem: Helmet, CORS, Rate-limit, Multipart, Sensible nativos
- Async/await nativo: Sem callback hell, error handling via try/catch

Prisma ORM

- Type-safe: Queries tipadas end-to-end (schema != tipos TS)
- Migrations: Versionamento de schema, rollback seguro
- Relations: Type-safe joins, include/select tipados
- Performance: Query optimization, connection pooling, prepared statements
- Developer experience: Prisma Studio, autocomplete, type hints

Zod Validation

- Schema-first: Validação declarativa, inferência de tipos automática
- Runtime + compile-time: Valida em runtime, tipa em compile-time
- Composability: .extend(), .partial(), .pick(), .omit() reutilizáveis
- Fastify integration: fastify-type-provider-zod integra nativamente

JWT + HttpOnly Cookies

- HttpOnly: Imune a XSS (JavaScript não acessa)
- SameSite=Strict: CSRF protection nativo
- Secure: Apenas HTTPS em produção
- Short expiry + refresh token pattern (futuro): Segurança em camadas

Sharp Image Processing

- WebP output: 30% menor que JPEG, qualidade visual equivalente
- Resize on upload: 800px max, previne uploads gigantes
- Metadata stripping: Remove EXIF/GPS (privacidade)
- Pipeline streaming: Baixo uso de memória

4. BANCO DE DADOS - POSTGRESQL + PRISMA

Modelos Principais

```
// User - Admin do painel
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String
  name        String
  role        Role      @default(ADMIN)
  lastLoginAt DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@index([email])
}

enum Role { ADMIN CUSTOMER }
```

```

// Store - Configuração da loja
model Store {
  id          String    @id @default(cuid())
  name        String
  whatsappNumber String
  email        String?
  phone        String?
  address      String?
  businessHours Json?
  socialLinks  Json?
  categories   Category[]
  settings     StoreSettings?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}

// Category - Categorias de produtos
model Category {
  id      String    @id @default(cuid())
  name    String    @unique
  label   String
  storeId String
  store   Store     @relation(fields: [storeId], references: [id], onDelete: Cascade)
  products Product[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([storeId])
}

// Product - Produtos do catálogo
model Product {
  id      String    @id @default(cuid())
  name    String
  price   Decimal   @db.Decimal(10, 2)
  image   String
  details String?
  categoryId String
  category Category @relation(fields: [categoryId], references: [id], onDelete:
Cascade)
  active   Boolean   @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([categoryId])
  @@index([active])
}

// LeadEvent - Tracking de intenção de compra
model LeadEvent {
  id          String    @id @default(cuid())
  productId   String
  productName  String
  productPrice Decimal   @db.Decimal(10, 2)
  eventType   EventType
  sessionId   String
  ipAddress   String?
  userAgent   String?
  referrer    String?
  createdAt   DateTime   @default(now())
  @@index([productId])
  @@index([eventType])
  @@index([sessionId])
  @@index([createdAt])
}

```

```
enum EventType { PRODUCT_VIEW BUY_CLICK WHATSAPP_REDIRECT }
```

Por que PostgreSQL?

- ACID compliance: Transações confiáveis, consistência garantida
- JSONB: Flexibilidade para dados semi-estruturados (businessHours, socialLinks)
- Extensões: pg_trgm (busca textual), uuid-osp, citext
- Confiabilidade: 25+ anos, usado por empresas críticas
- Escalabilidade: Partitioning, read replicas, connection pooling (PgBouncer)

5. SEGURANÇA IMPLEMENTADA

SQL Injection	Prisma ORM (parametrized queries)	Impossível via ORM
XSS	HttpOnly cookies + CSP + React escape	Token inacessível via JS
CSRF	SameSite=Strict + Origin check	Requests cross-origin bloqueados
Input Validation	Zod schemas em todos endpoints	Dados inválidos rejeitados cedo
Rate Limiting	@fastify/rate-limit (100 req/min/IP)	DoS/Brute force mitigado
Headers	Helmet + CSP restritivo	Clickjacking, MIME sniffing, XSS
File Upload	Sharp reprocessa + MIME validation	Polyglots, EXIF, scripts removidos
Secrets	Env vars + .gitignore	Nenhum secret no Git
Auth	JWT + bcrypt (12 rounds) + HttpOnly	Session hijacking difícil

6. ADMIN PANEL -
FUNCIONALIDADES

Auth	Login, logout, sessão persistente	JWT + HttpOnly cookie, ProtectedRoute
Dashboard	KPIs, produtos recentes	Stats cards, tabela paginada
Produtos	CRUD completo, busca, paginação	Modal form, validação Zod, status toggle
Categorias	CRUD, contagem produtos	Modal create/edit, delete com validação
Analytics	KPIs, top produtos, filtro data	Date range, conversion rate badges, tabela ordenável

Componentes Reutilizáveis

- AdminLayout: Sidebar colapsável, logo, navegação ativa, logout
- ProductForm: Modal criar/editar, select categoria, preview imagem, validação
- CategoriesPage: Tabela, modal criar, busca, delete com validação
- AnalyticsPage: Filtro data, 4 KPIs cards, tabela top 10 com badges coloridos

7. LEAD TRACKING &
ANALYTICS

Eventos Rastreados (useProductModal)

- PRODUCT_VIEW: Usuário abre modal do produto
- BUY_CLICK: Usuário clica "Quero comprar"
- WHATSAPP_REDIRECT: Usuário clica "Comprar pelo WhatsApp"

Payload Enviado para /api/lead-events

```
{
  productId: "uuid",
  productName: "Camisa Lacoste",
  productPrice: 90.00,
  eventType: "PRODUCT_VIEW" |
  "BUY_CLICK" | "WHATSAPP_REDIRECT",
  sessionId: "uuid-
v4", // sessionStorage
  referrer: "https://
google.com/", // document.referrer
  ipAddress:
  "auto", // backend
  userAgent:
  "Mozilla/5.0..." // backend
}
```

Analytics Dashboard

- 4 KPIs: Views, Cliques, WhatsApp Redirects, Taxa Conversão
- Filtro por data (início/fim)
- Top 10 produtos: Views, Clicks, WhatsApp, Conversão %
- Badges coloridos: Verde ("e5%), Amarelo (2-5%), Vermelho (<2%)

8. PWA (PROGRESSIVE WEB APP)

- vite-plugin-pwa: registerType: "autoUpdate"
- Service Worker: generateSW (Workbox)
- Manifest: name, icons (72-512px), theme_color #0ea5e9, display: standalone
- Runtime Caching (Workbox):
 - - Google Fonts: CacheFirst (1 ano)
 - - API: NetworkFirst (24h, timeout 10s)
 - - Imagens produtos: CacheFirst (30 dias)
- Precache: ~68 entries (~2.9 MB)
- Ícones: 8 tamanhos (72-512px) em public/icons/

9. DEVOPS & CI/CD

Docker API	server/Dockerfile	Multi-stage: deps !' builder !' runner (Alpine, non-root)
Docker Web	Dockerfile	Multi-stage: deps !' builder !' runner (nginx/preview)
Docker Compose	docker-compose.yml	Postgres 16 + Redis 7 + API + Web, healthchecks
CI/CD	.github/workflows/ci.yml	lint !' typecheck !' build (Node 20, ubuntu-latest)
Pre-commit	.husky/pre-commit	lint-staged (ESLint + Prettier)
Prisma	prisma/schema.prisma	Migrations versionadas, seed.ts
Scripts	package.json	dev, build, lint, format, test, prisma:*

10. ESTRUTURA DE PASTAS COMPLETA

```
Fabulosa-Ecommerce/
% % % docs/ # 5
PDFs técnicos
% % % public/
% % % icons/ # 8
ícones PWA (72-512px)
% % % uploads/ #
Imagens upload (Sharp WebP)
% % % server/ #
Backend Fastify
% % % prisma/
% % % schema.prisma #
Models completos
% % % seed.ts #
Seed: store, 2 cats, 32 produtos, admin
% % % src/
% % % config/index.ts # Env
validation (Zod)
% % % controllers/ #
Auth, Product, Category, LeadEvent
% % % middleware/ #
auth (JWT cookie+header), errorHandler
% % % repositories/ #
Prisma queries encapsuladas
% % % routes/ #
index.ts + upload.ts
% % % services/ #
Business logic (separation of concerns)
% % % schemas/ # Zod
validation schemas
% % % types/ #
Types compartilhados
% % % utils/ #
logger, prisma client singleton
% % % index.ts # App
entry point
% % % Dockerfile #
Multi-stage Alpine
% % % docker-entrypoint.sh #
Migrate + seed on start
% % % package.json
% % % src/ #
```

```

Frontend React 19 + Vite 7
  % % % components/
  % % % admin/
Admin panel completo
  % % % layout/
Navbar, Footer
  % % % sections/
Hero, About, ProductCatalog
  % % % ui/
ProductCard, ProductModal
  % % % contexts/admin/
AuthContext (HttpOnly cookies)
  % % % data/collections.ts
Dados tipados (fallback)
  % % % hooks/
useProductModal, useCollections,
useScrollToSection
  % % % lib/api.ts
API client (credentials: include)
  % % % types/
api.ts, index.ts
  % % % App.tsx
Routes: / + /admin/*
  % % % main.tsx
  % % % tests/
Testes
  % % % e2e/
Playwright
  % % % unit/
Vitest
  % % % docker-compose.yml
Postgres + Redis + API + Web
  % % % Dockerfile
Frontend multi-stage
  % % % .github/workflows/ci.yml
Lint !' Typecheck !' Build !' Test
  % % % .husky/pre-commit
lint-staged
  % % % eslint.config.js
Flat config TS + React
  % % % .prettierrc
Code style
  % % % tsconfig.json
Strict TS + path aliases
  % % % vite.config.ts
React + Tailwind + PWA
  % % % package.json

```

11. COMANDOS ESSENCIAIS

Desenvolvimento Local

```

# Terminal 1 - API
cd server
cp .env.example .env
# Configure DATABASE_URL, JWT_SECRET
docker compose up -d postgres redis
npm run prisma:migrate
npm run prisma:seed
npm run dev

# Terminal 2 - Frontend

```

```
cd ..
npm run dev
```

Docker (Produção/Staging)

```
docker compose up -d --build
# Frontend: http://localhost:5173
# API: http://localhost:3000
# Admin: http://localhost:5173/
admin/login
# Demo: admin@fabulosamodas.com /
admin123
```

Testes

```
# Frontend
npm run test           # Vitest
watch
npm run test:run       # Vitest CI
npm run test:coverage  # Coverage
report

# E2E
npm run test:e2e       # Playwright
npm run test:e2e:ui    # Playwright
UI

# Backend
cd server
npm run test           # Vitest
watch
npm run test:run       # Vitest CI
```

11. DECISÕES TÉCNICAS CHAVE (ADR-STYLE)

Frontend Framework	React 19	Vue 3, Svelte	Ecosistema maduro, equipe familiar, Concurrent features
Build Tool	Vite 7	Webpack, Next.js	Dev server instantâneo, ESM nativo, plugin ecosystem
Language	TypeScript Strict	JavaScript, Flow	Type safety, refactoring seguro, DX superior
Styling	Tailwind CSS 4	CSS Modules, Styled Components	Utility-first, bundle pequeno, design system nativo
Backend Framework	Fastify	Express, NestJS	Performance 2x, validação nativa, TS-first
Database	PostgreSQL	MySQL, MongoDB	ACID, JSONB, extensões, confiabilidade enterprise
ORM	Prisma	TypeORM, Drizzle, Knex	Type-safety end-to-end, migrations, DX
Validation	Zod	Joi, Yup, class-validator	Type inference, composability, Fastify integration
Auth	JWT + HttpOnly Cookie	localStorage, Session	XSS-proof, CSRF-proof, mobile-friendly
Image Processing	Sharp	JimP, ImageMagick	Performance, WebP, streaming, metadata stripping
Testing	Vitest + Playwright	Jest + Cypress	Vite-native, ESM, faster, modern API
PWA	vite-plugin-pwa	Workbox manual, next-pwa	Integrado Vite, auto-update, Workbox abstraction
Container	Docker Multi-stage	VM, Serverless	Reprodutível, leve, cache layers, security
CI/CD	GitHub Actions	GitLab CI, CircleCI	Gratuito p/ público, integração nativa GH

12. PRÓXIMOS

PASSOS (ROADMAP)

- 1. Banco PostgreSQL real (Neon/Supabase/Railway) + DATABASE_URL
- 2. JWT_SECRET forte (openssl rand -base64 32) para produção
- 3. Domínio real + HTTPS + CORS_ORIGIN no .env
- 4. Testes: Vitest (unit), Playwright (E2E), Coverage >80%
- 5. Sentry (error tracking) + Pino structured logs
- 6. Deploy: Railway/Render (API) + Vercel/Netlify (Web) + PostgreSQL managed
- 7. Phase 6: WhatsApp Lead Tracking completo + Analytics avançado
- 8. Phase 7: Image upload no ProductForm (dropzone + preview + Sharp)
- 9. Phase 8: Wishlist, compartilhamento, comparação produtos
- 10. Phase 9: Testes automatizados + CI/CD deploy automático